# INNOPOLIS UNIVERSITY

## SNE MASTER TRACK

## 2025

## COURSE: [F25] COMPUTER SYSTEMS AND NETWORKS

**CSN LAB 2 REPORT (PART A)**

**On**

**Architecture Patterns**

**Student Name: HASIB AL TAHSIN**

**Application ID: 187623**

**Email:** h.tahsin@innopolis.university, hasibjust10@gmail.com

**Submission Date: 18.09.2025**

# PART A

## Task 1

### 1.1. Provide 3 unique definitions for software architecture.

**Answer:**

**Logical View-Focused Definition:**

Software architecture is a form of system that primarily represents the code structure of the software to meet functional requirements. Here, the system is organized into classes, objects, subsystems, and their relationships, so that it is understood which parts perform which responsibilities and how they cooperate with each other. The basic basis for how user tasks will be implemented is determined through this Logical View. For example - Online Library Management System **[1], [2].**

**Development View-Focused Definition:**

Also known as the Module Organization View, a framework view of software architecture that focuses on the organization of modules, their interdependencies, and code-based separation. These frameworks and decisions form the basis for managing the development, maintenance, and expansion of the software **[3].**

**Physical View-Focused Definition:**

A view of software architecture that determines how the physical deployment and distribution of system components - machines, servers, network connections, etc. - will be done. It shows which software components will run on which hardware or nodes, how they will be connected to each other, and how the network topology, communication links, and bandwidth requirements will be met **[1].**

**1.2. List and explain with sample diagrams, 3 types of software architecture.**

**Answer:**

1. **Layered (N-tier) Architecture**

   Such systems are divided into multiple layers, where being layer-based, the system can be easily maintainable and easily tested **[3], [4].**

**Diagram:**

[ End User ]

↓

[ UI Layer ]

↓

[ Business Logic Layer ]

↓

[ Data Access Tier ]

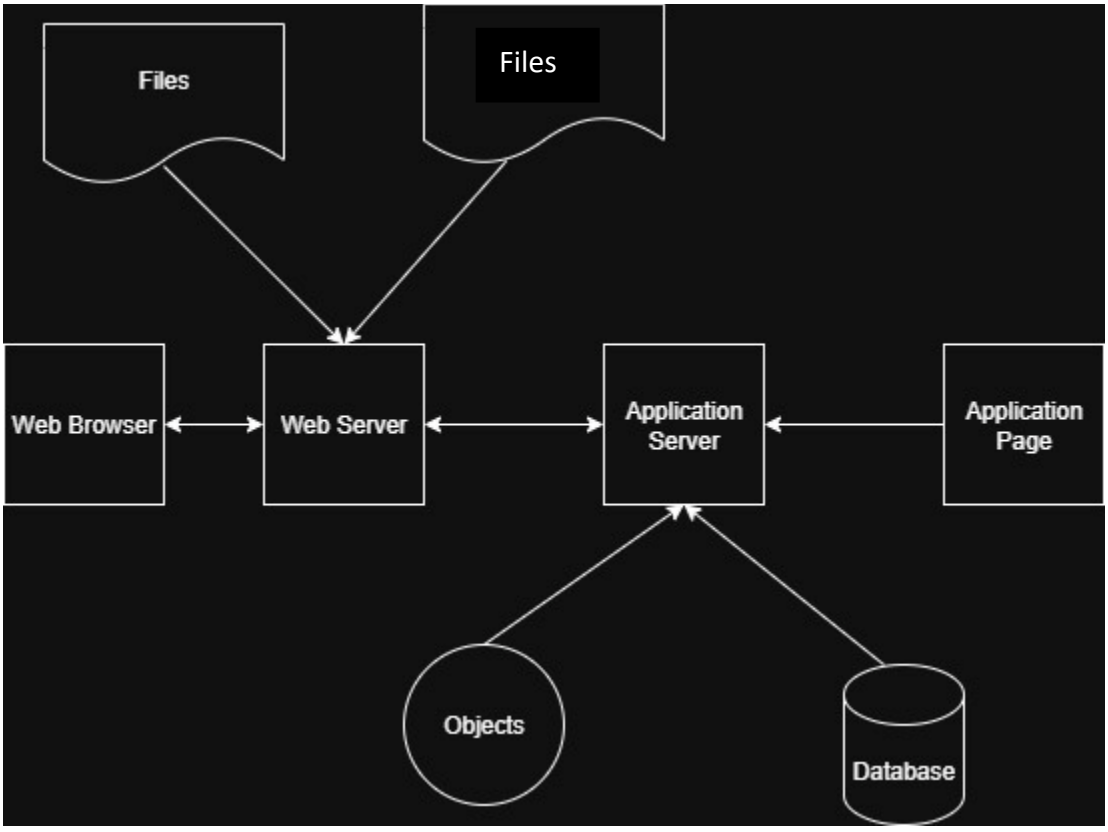
↓

[ Data Store ]

## 2. Physical/Deployment View-Focused Software Architecture

In this type of software architecture, the Physical View-Focus shows the placement and network distribution of system components on physical nodes and determines which server or client will run which component, node connectivity, and topology **[1], [5].** One of the advantages of this technology is its high performance and easy scalability. In addition, potential network and hardware-related problems can be predicted in advance, which makes management more effective. However, it also has some limitations. In the case of complex network systems, configuration becomes quite difficult and changes in deployment have to be adjusted accordingly.

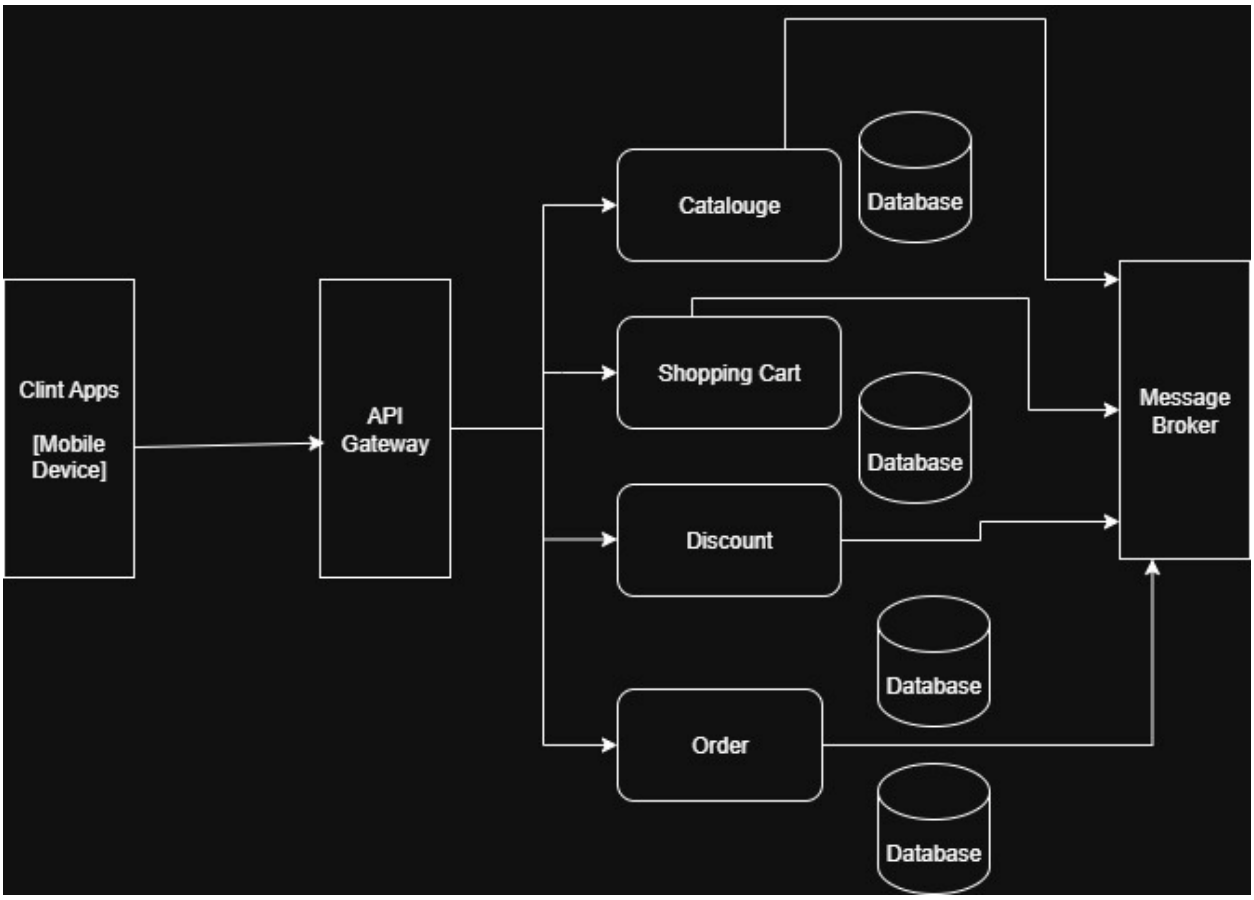
**Diagram:**



### 3. Microservices Architecture

This type of software architecture divides a large system into smaller, independent services. Each service is deployable separately and can have different functionality. This type of software architecture is easy to scale and allows specific services to be updated separately **[6], [7].**

**Diagram:**



**1.3 What are the differences between System architecture and Software architecture?**

**Answer:**

**Difference between System Architecture and Software Architecture**

Two important disciplines in software engineering and system design are System Architecture and Software Architecture. Below is the difference between System Architecture and Software Architecture:

| System Architecture | Software Architecture |
|---|---|
| System Architecture is the overall structure of a larger system where hardware, software, and other infrastructure components come together to form an integrated system **[8].** | Software Architecture is the structure of a software system, which deals with classes, modules, subsystems, and their relationships and interfaces **[9].** |
| The components of the system are hardware, software, network, etc. | Software components are its interface, logical structure, and user functionality. |
| Its main goal is to operate a system effectively, reliably, and scalable. | The software architecture determines and ensures maintainability, flexibility, and scalability. |
| The structures of System Architecture are Physical view, hardware topology, network topology, communication channels. | The frameworks of Software Architecture are Logical view, module decomposition, component interactions. |

**1.4 Differentiate between the following in the context of software architecture and provide examples;**

- **Software correctness and Software robustness.**
- **Topic & queue**

- **Architecture & design**
- **User: primary stakeholder and secondary stakeholder**
- **Cohesion & coupling.**
- **Software Correctness and Software Robustness**

**Answer:**

| Software Correctness | Software Robustness |
|---|---|
| Correctness is a measure of whether a software system is working according to a "specific specification" or functional requirements **[10].** | Robustness is the extent to which software can continue to function in "unexpected" or unimaginable situations, such as incorrect inputs, environmental problems, stress, or constraints **[11].** |
| Only useful for specified situations and inputs, such as normal use cases, without any errors **[11].** | It has the ability to behave "reasonably" even in environments with incorrect, undefined inputs, or incomplete/partial errors, going beyond specified conditions **[11].** |
| Checking whether the output matches the specified unit tests, formal verification, assertions, or specifications. | It has the ability to behave "reasonably" even in environments with incorrect, undefined inputs, or incomplete errors, even outside of specified conditions. |
| For example, if a calculator can perform correct addition, subtraction, etc. on all numbers as specified in the specification, then it is correct **[12].** | For example, if a calculator does not crash when it receives invalid or non-numeric input, but instead gives an appropriate error message or shows default behavior, then it is robust **[11].** |

- **Software Architecture Topic and Queue**

| Topic | Queue |
|---|---|
| It uses the Publish/Subscribe (Pub/Sub) pattern. A "Topic" is a channel where a message can be published and received by multiple subscribers simultaneously **[13].** | It uses a Point-to-Point (P2P) model. A "Queue" is a channel where each message is received by only one consumer **[14].** |
| In this one-to-many or Producer, if a message is sent, many Consumers can receive it. | In one-to-one, each message sent by the Producer can only be consumed by one Consumer. |
| Order_Placed: This message can be received by the Inventory Service, Notification Service, and Analytics Service together. | Payment_Queue: Each Payment Job will be processed by only one Payment Worker. |

- **Software Architecture and Design**

| Architecture | Design |
|---|---|
| In Software Architecture, very high levels are usually chosen, dividing the major parts of the system, the relationships between components, non-functional attributes, etc. **[15].** | This part deals with low-level, everyday details - classes, interfaces, algorithms, code structures, etc. **[15].** |
| Its main goal is to prioritize non-functional aspects such as stability, scalability, security, performance, maintainability, etc. | Its main goal is to determine functional behavior, how use cases will work, how code will be written, what design patterns or structures will be used, etc. **[16].** |
| A web application where I am deciding: will it have two services (frontend and backend), login service, database server, server-less or cloud, etc. | The design of that backend service is: what classes will there be, how will the repository pattern be implemented, what API interfaces will there be, what error handling will be like, etc. |

- **Software Architecture User: Primary Stakeholder and Secondary Stakeholder**

| Primary Stakeholder | Secondary Stakeholder |
|---|---|
| In software architecture, primary stakeholders are usually those who directly participate in or are directly affected by the project (use, maintenance, decision) – such as end-users, clients/customers, project sponsors, developers, etc. **[17].** | Secondary stakeholders refer to those who are not directly involved in the project, but are partially affected or may be affected by the project's outcomes - such as the media, competing companies, government regulators, the general public, etc. **[17].** |
| They are directly involved in requirements definition, decision making, and usage. They are most needed in the core functional and non-functional requirements of the project **[17].** | They participate in the project's reputation, social or ethical impact, regulations, external demands, etc., although they do not directly participate in the decision-making process **[17].** |
| Example: In the case of a mobile app: the app user, the app buyer/company that is creating the app (Sponsor), the development team, etc. are the primary stakeholders **[17].** | Example: How the media will review the entire product, government privacy laws that may affect the app, competing companies – secondary stakeholders **[17].** |

- **Software Architecture Cohesion and Coupling**

| Cohesion | Coupling |
|---|---|
| In software architecture, cohesion determines how closely the components within a module (or class, or subsystem) work together to accomplish a specific task or purpose **[18].** | Coupling basically refers to the degree of dependence and connection between different modules or components, or how much one module depends on another **[18].** |
| High cohesion makes the system more clean, testable, and maintainable **[18].** | Low coupling makes the system easier to modify, add new features, and scale **[18].** |
| A "User Authentication" module that only has the functions of login, logout, password reset, etc. | If the "User Authentication" module interacts with too much other module code (Profile, Notification, Database Server), sudden changes can break other modules. |

# Task 2

## 2.1. List and explain the key concepts of Object-Oriented programming

**Answer:**

Object-Oriented Programming is a programming paradigm based on the concept of objects, in which a software system is modeled as real-world objects and their interactions. In this paradigm, data (attributes / state) and work (methods / behavior) are combined in the same place within an object **[19].**

| Key Concepts | Description | Impact |
|---|---|---|
| **Abstraction** | Highlighting important or useful features while hiding unnecessary or cluttered information. That is, the internal workings (implementation) of an object or class are kept "hidden" and the external user only knows what will work (method signatures, interface, etc.) **[20].** | This reduces complexity, meaning the user (recipient) does not need to know about the internal changes, making security and maintenance easier **[20].** |
| Class ও Object | A class is a blueprint for an object, containing its state (data/attributes) and behavior (methods). An object is a real instance of the class that implements those data and methods. Example: Car class, myCar object **[21].** | Organization of code where behavior and data are in one place and multiple objects can be created from the same class. |
| Polymorphism | The same interface or method name can be used in different contexts. For example: method overloading (same name, different parameters) or overriding (modifying a parent class method in a subclass) etc. **[20].** | The code is flexible and new behaviors can be added with fewer code changes. This makes it easier to use and extend. |
| Modularity / Reusability | Code is divided into small, independent classes and modules that are reusable. The code parts are loosely connected and have high cohesion [**22**]. | Improved maintenance and work sharing can be achieved. This speeds up development and reduces the impact of changes. |

| | | |
|---|---|---|
| Cohesion and Coupling | Cohesion is the relationship between the internal components of a module or class and the extent to which they are connected to a common task or responsibility. Coupling is the degree of dependency between modules-classes, where changes to one part affect the other parts **[12].** | High cohesion and low coupling make code more maintainable, reusable, and testable. Conversely, code becomes complex and fragile **[20].** |
| Encapsulation | An object's data and behavior are encapsulated in a single unit (class) and data is not directly accessible from the outside, so instead getters/setters or public interfaces are used **[23].** | There is less internal state control and less external interference within the class. This increases system stability and reduces side effects when updating modules or objects **[23].** |

**2.2. What is an architectural view? Explain characteristics of architecture patterns in pattern based.**

**Answer:**

**Architectural view**

Architectural views are a way of presenting software architecture from different perspectives, where each view is organized around a specific stakeholder concern or need. Rather than showing all aspects of the entire system at once, different aspects (such as functionality, runtime behavior, infrastructure deployment, etc.) are presented separately. In this way, views facilitate decision-making, maintenance, and change planning **[24].**

**Key Aspects of Architectural Views**

- Each view addresses the needs of a specific stakeholder or group (for example, developers, operations team, users, managers) **[25].**

- Views typically display components and their connections (components and connectors), demonstrating static or dynamic behavior **[25].**
- Different views are related to each other, but each view only shows a part of its concern **[25].**
- Views can be created using various UML diagrams, deployment diagrams, sequence diagrams, state machines, etc. **[25].**

**Architectural Pattern**

An architectural pattern is a reusable, creative, and tested structural solution that is applicable in a specific context to solve a common problem. It refers to a design blueprint **[26].**

**Characteristics of Architectural Patterns**

Here are some of the key characteristics of architectural patterns:

- **High-Level Structure**

  Patterns typically define the large-scale connections and layouts of modules, subsystems, and infrastructure of the entire system. For example, layered architecture, client-server, microservices, etc. **[26].**

- **Context Dependence**

  Not every pattern will be suitable everywhere, the choice of pattern will depend on the system's learning, non-functional requirements (scale, performance, security, etc.), environment, and team capabilities **[27].**

- **Reusable / Tested Solution**

  Patterns are typically tested and used in many projects. Therefore, they work well for specific types of problems and constraints **[26].**

- **Ease of planned changes and maintenance**

  Patterns are designed in such a way that features can be added in the future, scaled, and changed without breaking the entire system. That is, flexibility and maintainability are good **[27].**

- **Addressing Stakeholder Concerns**

  When choosing a pattern, it is seen what concerns the stakeholders (users, operations, developers, security team) have and how well the pattern can address those concerns **[28].**

- **Attention to non-functional qualities**

  such as performance, reliability, security, fault tolerance, scalability, etc. should be consistent with **[27].**

**2.3. Which are the main approaches to describing a software architecture? List and explain with sample diagrams.**

**Answer:**

Below are the main approaches to describing Software Architecture:

1. **Layered Architecture:**

   The system is divided into different layers - typically a Presentation or UI layer, a Business Logic layer, and a Data Access or Persistence layer. Each layer has a distinct responsibility and the communication between the layers is clearly defined. This approach makes the system more organized and maintainable **[27], [29].**

**Diagram:**

[ End User ]

↓

[ UI Layer ]

↓

[ Business Logic Layer ]

↓

[ Data Access Teir ]

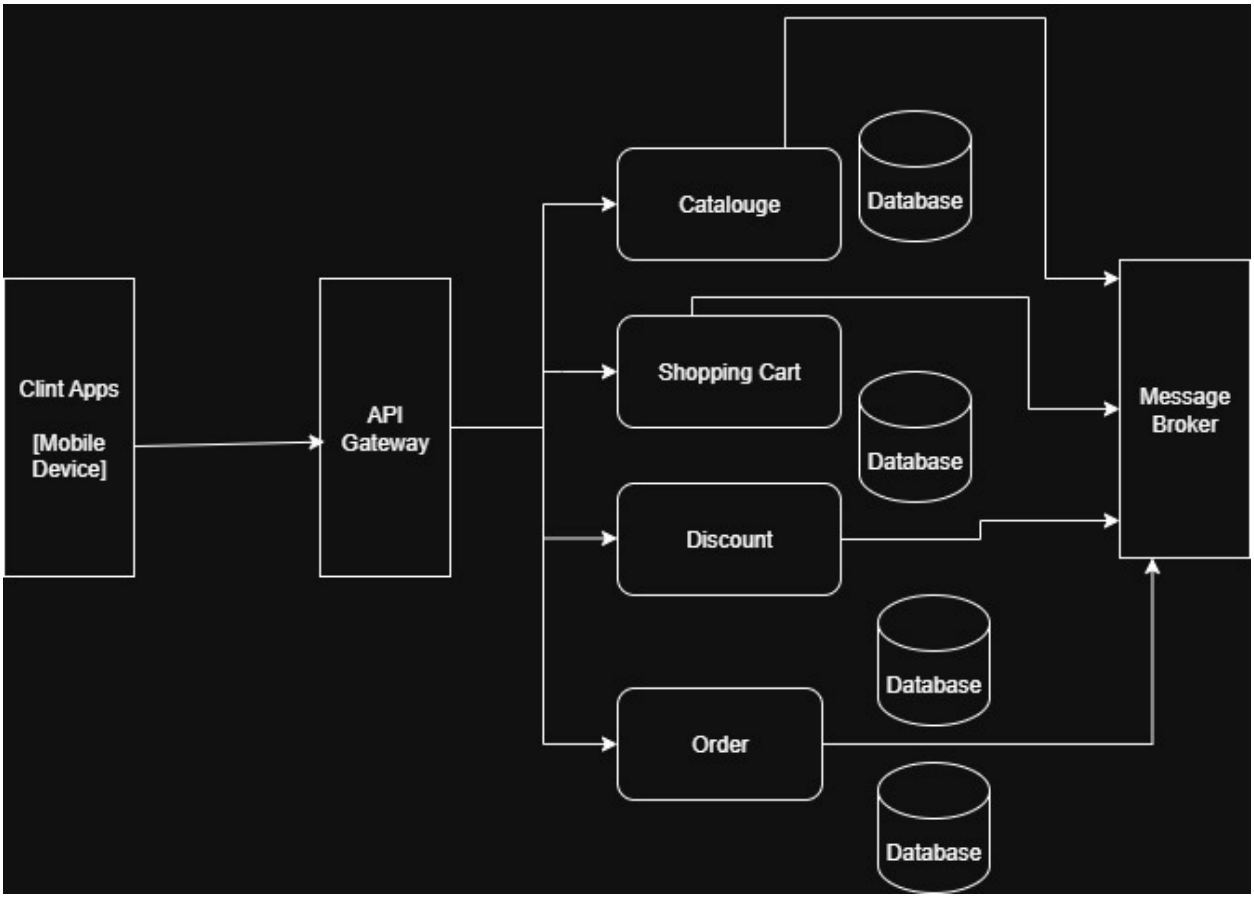
↓

[ Data Store ]

2. **Microservices Architecture:**

The entire application is divided into small, independent services. Each service performs a specific business task, is separately deployable, and is scalable. Communication between services can be API-based or message-based. Their work patterns can also be different. This type of software architecture is easy to scale and allows specific services to be updated separately **[1], [28], [30], [31].**

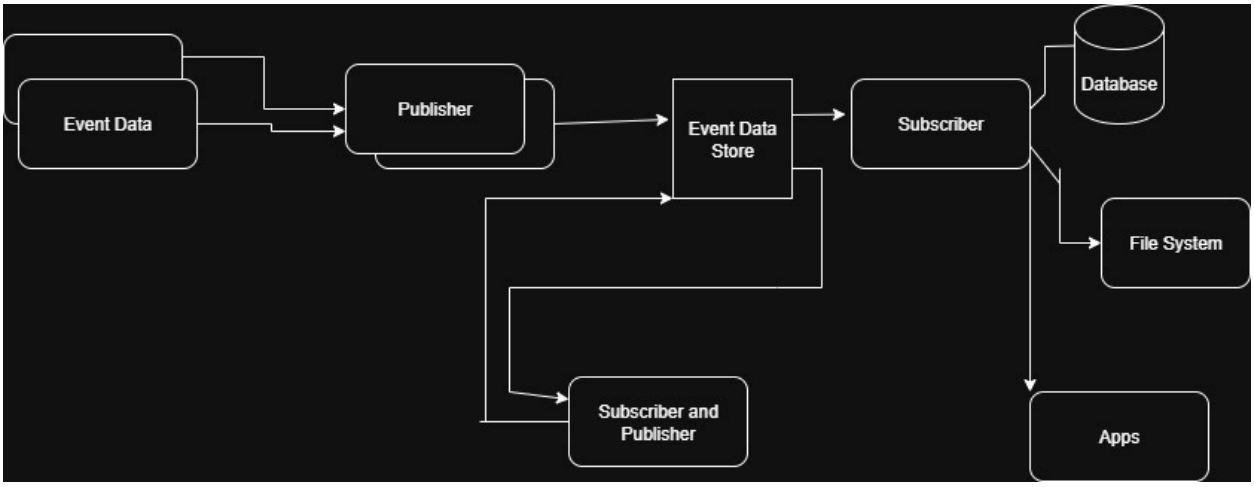
**Diagram:**




### 3. Event-Driven Architecture (EDA)

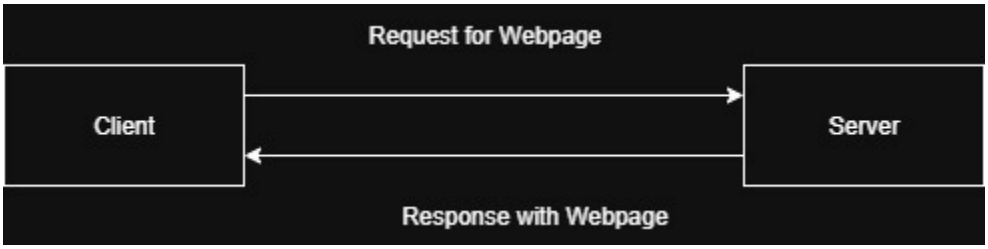
The workflow of the system is determined by the generation and reception of events. One component will generate the event, while others will listen and respond to it. Asynchronous processes and real-time responses are used in important cases [1], [32].

**Diagram:**




4. **Client-Server Architecture**

There is a client (User Interface / Client Application) that sends requests to the server and the server processes the data or provides the service. This approach to system architecture is common in network-based applications **[1], [33].**



**2.4. Provide the advantages and disadvantages of the main approaches explained in question 2.3.**

**Answer:**

1. **Layered Architecture (N-Tier / Layered Style)**

**Advantages:**

One of the advantages of this structure is that it is easy to change and maintain. Since each layer is separate, changes to one layer have relatively little impact on the other layers. It also provides advantages in testing, as testing can be done layer-wise. For example, the Business Logic Layer can be unit tested separately. Moreover, the organization of the code is also clear, so new developers can easily understand the entire structure **[1].**

**Disadvantages:**

Having more layers can have a performance impact because of the increased amount of communication between layers. In addition, if the layer boundaries are not properly defined, code can become overly dependent on each other or tightly coupled. This can create unnecessary overhead for small projects **[1].**

2. **Microservices Architecture**

**Advantages:**

This kind of architecture allows each service to be deployed and scaled independently, making the system more flexible. It is also technically advantageous, as each service can use a different tech stack. This allows large teams to work on multiple services at the same time, which increases overall efficiency **[34].**

**Disadvantages:**

A major limitation of this architecture is that it is quite complex to coordinate services. Also, maintaining data consistency is a major challenge. Moreover, DevOps activities and monitoring require additional time and cost, which makes this management more complex **[34].**

3. **Event-Driven Architecture (EDA)**

**Advantages:**

Event-based processing makes the system more responsive and scalable. In addition, the separate Producer and Consumer components create a loose coupling structure. This allows new events to be easily added or integrated into the system, making it more flexible **[35].**

**Disadvantages:**

In event-based systems, the order of events and data consistency can sometimes create problems. In addition, the debugging and testing process becomes relatively difficult. Moreover, the use of failover and fallback mechanisms to ensure uninterrupted functionality becomes very important in these systems **[35].**

4. **Client-Server Architecture**

**Advantages:**

This architecture allows data and logic to be centrally controlled by the server, ensuring centralized control. Being server-centric makes it relatively easy to maintain security and control permissions. It also provides advantages in maintenance and upgrades because the client and server can be updated separately **[1].**

**Disadvantages:**

One of the major limitations of this type of architecture is that if a server fails, the entire system can be shut down, i.e. it is vulnerable to Single Point of Failure. In addition, performance can be

degraded due to network latency and connectivity issues. Scaling to large systems is also a major challenge of this architecture **[1].**

## References


**[1]** https://www.geeksforgeeks.org/software-engineering/types-of-software-architecture-patterns/

**[2]** https://en.wikipedia.org/wiki/4%2B1_architectural_view_model

**[3]**https://www.utm.mx/~caff/doc/OpenUPWeb/openup/guidances/concepts/software_architecture_59A08DE0.html

**[4]** https://www.c-sharpcorner.com/article/micro-services-vs-n-tier-architecture/

**[5]** researchgate.net/figure/Physical-View-of-the-Architecture-of-a-Web-Application_fig2_2379715?__cf_chl_rt_tk=Dd0rcKX.XfestTiINcaHVNEUR5rVPhCv7iwo2geaWc8-1758058005-1.0.1.1-gk9qniickGkPi1lgmjhbS57FcW2KTrwCkkK7MCHZxBI

**[6]** https://webandcrafts.com/blog/what-is-microservices-architecture

**[7]** https://martinfowler.com/articles/microservices.html

**[8]** IEEE Std 1471-2000, "*Recommended Practice for Architectural Description of Software-Intensive Systems."* Available: https://ieeexplore.ieee.org/document/888574

**[9]** Bass, Len, Paul Clements, Rick Kazman. *Software Architecture in Practice*, 3rd Edition, Addison-Wesley, 2012. Available: https://www.sei.cmu.edu/reports/12tr024.pdf

**[10]** https://carver.cs.ua.edu/Data/Journals/SLR-DesignPatterns/attributes.html

**[11]** https://ics.uci.edu/~dfredmil/ics121-FQ99/lecture/two/tsld007.html

**[12]** https://accu.org/journals/overload/17/93/wilson_1585/

**[13]** https://kafka.apache.org/documentation/

**[14]** https://docs.oracle.com/javaee/7/tutorial/jms-concepts002.htm

**[15]** https://en.wikipedia.org/wiki/Architecture_description_language

**[16]** https://www.geeksforgeeks.org/difference-between-software-design-and-software-architecture/

**[17]** https://www.engati.com/glossary/cohesion-and-coupling

**[18]** https://www.engati.com/glossary/cohesion-and-coupling

**[19]** https://en.wikipedia.org/wiki/Object-oriented_programming

**[20]** https://www.geeksforgeeks.org/understanding-encapsulation-inheritance-polymorphism-abstraction-in-oops/

**[21]** https://en.wikipedia.org/wiki/Object-oriented_programming

**[22]** https://ducmanhphan.github.io/2020-01-31-Some-important-concepts-of-OOP/

**[23]** https://www.computersciencebytes.com/object-oriented-programming/fundamental-concepts-of-oop/

**[24]** https://home.iscte-iul.pt/~hro/RUPSmallProjects/core.base_rup/guidances/concepts/software_architecture_4269A354.html

**[25]** https://home.iscte-iul.pt/~hro/RUPSmallProjects/core.base_rup/guidances/concepts/software_architecture_4269A354.html

**[26]** https://blog.sivo.it.com/software-architecture-patterns/what-is-an-architectural-pattern-in-software-architecture/

**[27]** https://cs.ccsu.edu/~stan/classes/CS410/Notes16/06-ArchitecturalDesign.html

**[28]** https://pubs.opengroup.org/architecture/togaf7-doc/arch/p4/patterns/patterns.htm

**[29]** https://www.c-sharpcorner.com/article/micro-services-vs-n-tier-architecture/

**[30]** https://webandcrafts.com/blog/what-is-microservices-architecture

**[31]** https://martinfowler.com/articles/microservices.html

**[32]** https://hazelcast.com/foundations/event-driven-architecture/event-driven-architecture/

**[33]** https://www.scaler.com/topics/nodejs/client-server-architecture/

**[34]** https://martinfowler.com/articles/microservices.html

**[35]** https://www.ibm.com/docs/en/architecture?topic=architecture-event-driven